Rectangular Cancellation Without Search

Stopping-set certificates, exact passive transport,
and deferred-handle kernels for unknot recognition

Research continuation prepared for Vladimir Reshetnikov's ProveIt project
Developed with ChatGPT

12 October 2026

Abstract

The maintained normal-cut boundary kernel reduces a geometric cancellation problem to a finite nonnegative matrix, retaining binary bulk genus separately. Its general singleton planner repeatedly chooses a unit entry and performs a positive, saturated update. We prove that full elimination in this calculus requires no search and no fill: it is equivalent to private-column peeling, absence of a weighted stopping set, and existence of a row-spanning square minor of permanent one. The residual obstruction is canonical. A sparse producer and an independent timestamp checker require linear incidence work.

The result also corrects the interpretation of the current minimum-fill heuristic: its full-success decisions are complete for this calculus, and every successful run has zero fill. The improvement is therefore a scheduling and verification improvement, not increased topological coverage. For partial reduction we characterize the exact minimum number of deferred rows as a stopping-set transversal number, and obtain a complete small-deferral algorithm. Deferred rows are retained as genuine two-handles. Their exact occurrence counts have a finite path-sum formula and polynomial bit complexity, even when explicit splicing would be exponentially long.

A family of genuine three-ball handle structures has linearly growing deferral number. Thus small deferral cannot be a universal consequence of ball topology alone. Source-certified changes of the disk or attaching-curve system, rather than further positive-pivot search, are necessary to escape these obstructions.

The package includes 34 passing unit tests, an exhaustive 31,907-matrix audit, 1,241 optimal-deferral checks, reproducible paired measurements, and a hash-guarded integration proposal. No complete knot benchmark or production-checkout patch application was run. The results improve a local operator and identify its exact boundary; they do not assert a general quasi-polynomial unknot-recognition bound.

Reference repository revision

`8cb7d9d25ac8cb64a9d147304e3441e98e94c7a4`

Theorems are proved here; finite audits supplement, not replace, the proofs.

Contents

1 Research position and the concrete opportunity **2**
1.1 What is being improved . . . 2
1.2 Relation to Lackenby's target . . . 2
1.3 Prior ingredients and the novelty boundary . . . 2

2 The positive singleton calculus **3**

3 Stopping obstructions survive all positive pivots **4**

4 The rectangular completeness theorem **5**

5 What the native minimum-fill rule already decides **6**

6 Linear certificates and independent verification **7**
6.1 The producer . . . 7
6.2 A timestamp proof . . . 8
6.3 Backward-compatible native replay . . . 8

7 Implicit bundle columns and the topological conclusion **8**
7.1 Application to the maintained boundary kernel . . . 9

8 Optimal partial reduction by deferred rows **10**
8.1 Obstruction-directed branching and lower bounds . . . 11

9 Exact passive transport without expanded words **11**

10 A geometric barrier: balls with large stopping cores **13**
10.1 An even simpler unreduced-position warning . . . 15
10.2 Fibonacci amplification as a controlled example . . . 15

11 Executable evidence and controlled measurements **15**
11.1 Independent checks rather than a single implementation . . . 15
11.2 Paired producer and proof-query timings . . . 16

12 Integration plan and scope of the delivered changes **17**

13 Further research questions **18**

14 Conclusion **19**

A Source audit and reproducibility boundary **19**

B Claim and evidence ledger **20**

C Reproduction and minimal use **20**

1 Research position and the concrete opportunity

1.1 What is being improved

The inspected ProveIt implementation already has a source-authenticated finite boundary kernel for a supplied normal cut. Its geometric input is not an arbitrary presentation of a group. The construction retains core one-handles and named bundle-boundary disks, and represents certain other handles through binary counts. The maintained proofs explain how the finite matrix projects a geometric handle model and how the genus census is recovered after successful cancellations [8, 9].

The relevant numerical operation is unusually rigid. If a two-handle has exactly one intersection with a chosen one-handle cocore, the planner removes that row and column and updates all other intersection counts by a positive formula. Only the values zero, one and at least two are retained. The producer first attempts row-leaf cancellation and then uses a minimum-fill singleton heuristic [6, 7]. This raises a precise question:

> Does searching more positive-pivot orders ever improve a full-success decision beyond an elementary no-fill peeling test?

The answer is no. The correct rectangular test peels *private columns*, not only leaf rows. The proof explains both the completeness of the test and the otherwise surprising completeness of the native minimum-fill rule for this particular decision problem.

We then separate a genuinely different problem: how many rows can be eliminated when some rows must remain? Private-column peeling of the full matrix does not optimize that objective. Allowing a small explicitly retained set leads to a complete parameterized reduction and to exact compressed transport of its numerical incidence data.

1.2 Relation to Lackenby's target

Oxford's 2021 announcement gives the target $N^{c \log N}$ [1]. Lackenby's July 2026 preprint develops an incompressibility and hierarchy algorithm. Section 9 explicitly distinguishes iteration counts from still-unspecified operation costs, obtaining $L(g+1)^L$ in hierarchy length L and pattern complexity g. Its refined algorithm bounds hierarchy length by $4c_q(\mathcal{H})$; this is not a logarithmic bound in diagram size [2].

Our result concerns one finite cancellation operator inside that program. Reducing a local cubic scan to linear incidence work is useful even when source construction remains dominant. The small-deferral theorem gives a quasi-polynomial bound for a precisely specified local parameter regime. Neither fact supplies the missing all-input hierarchy estimates or a source-preserving method for finding every required geometric simplification.

1.3 Prior ingredients and the novelty boundary

Stopping sets and their order-independent peeling residual are classical in erasure decoding [3]. Triangular patterns and uniquely restricted matchings are also classical [4]. Deferral has an algorithmic analogy with inactivation decoding, where temporarily unresolved variables are carried symbolically [5]. We do not claim these general ideas as new.

The contribution developed here is the exact obstruction theorem for the *positive saturated geometric-pivot calculus*, its consequences for the inspected native heuristic and omitted columns,

the optimal-deferral interpretation, and exact passive-count transport with explicit bit bounds. The geometric counterfamily isolates an important limit of any small-stopping-core strategy. Historical priority for the entire combination has not been exhaustively established.

The incoming directory was inventoried, but two especially relevant archive bodies, `ProveIt_Unknot_Stopping_Cores_2026-10-12.zip` and `ProveIt_Saturated_Bundle_Cancellation_2026-10-12.zip`, could not be decoded through the available connector. They were not silently treated as reviewed. A merge should compare these manuscripts before assigning novelty or suppressing overlapping proofs. The maintained source files listed in Appendix A were inspected directly.

2 The positive singleton calculus

Let $A = (a_{rc})$ be a finite nonnegative integer matrix with row set R and column set C. Write $m = |R|$, $n = |C|$, and s for its number of nonzero entries. All zero rows are part of the input. Columns of all zeroes may be omitted numerically, but must still be counted when a geometric genus census requires them.

Define $\mathrm{sat}(x) = \min\{2, x\}$ and apply this map entrywise. A *positive singleton pivot* at (r, c) is legal when $a_{rc} = 1$. It deletes row r and column c, and on surviving indices performs

$$a'_{ih} = \mathrm{sat}(a_{ih} + a_{ic}a_{rh}). \tag{1}$$

A schedule is *full* when it deletes every row. Surviving columns are permitted. Unless explicitly stated otherwise, every numerical matrix below is saturated.

Lemma 2.1 (Saturation is exact for this decision language)**.** *Saturating the initial matrix and after every positive update preserves exactly which singleton schedules are legal. Moreover, for nonnegative numbers x_i,*

$$\sum_i \mathrm{sat}(x_i) = 1 \quad \Longleftrightarrow \quad \sum_i x_i = 1.$$

Proof. For nonnegative integers x, y,

$$\mathrm{sat}(x + y) = \mathrm{sat}(\mathrm{sat}(x) + \mathrm{sat}(y)), \qquad \mathrm{sat}(xy) = \mathrm{sat}(\mathrm{sat}(x)\,\mathrm{sat}(y)).$$

These identities follow by separating the cases zero, one, and at least two. Induction over a fixed sequence of updates proves that the saturated replay equals the saturation of the unsaturated replay. An entry is one in one replay exactly when it is one in the other. For the second assertion, a sum is one exactly when one summand is one and all other summands are zero. □

The plus sign is essential. Equation (1) is not Gaussian elimination over a field and is not an update of signed exponent sums. Later sections use an ordinary matrix inverse only as notation for a finite positive path sum; it must not be confused with a Schur complement for the original matrix.

Definition 2.2 (Private columns and stopping sets)**.** For $S \subseteq R$, a column is *private on S* when

$$\sum_{i \in S} a_{ic} = 1.$$

A subset S is a *weighted stopping set* when every column has total zero or at least two on S. The empty set is allowed as a stopping set; a *stopping obstruction* is nonempty.

A private column has a unique incident row, and its entry there is one. Deleting this row and column performs no update on any surviving row, because every other entry in the pivot column is zero. *Private-column peeling* repeats such deletions until none remains. The result is not the same as removing a row containing a single unit entry: a private-column row may contain arbitrarily many other entries.

Example 2.3 (The rectangular distinction)**.** The one-row matrix $(1 \;\; 1)$ has no leaf row but is fully cancellable. More generally, the $m \times (m+1)$ matrix $[I_m \mid \mathbf{1}]$ has no leaf row and has a private-column schedule. This family will also give an exact separation between repeated scans and sparse peeling.

3 Stopping obstructions survive all positive pivots

The key point is stronger than the usual assertion that stopping sets block peeling: they block *every* positive singleton schedule.

Lemma 3.1 (Positive-pivot obstruction invariant)**.** *Suppose S is a nonempty stopping set in the current saturated matrix. After a legal positive singleton pivot, the row identities from S that survive form a nonempty stopping set in the new matrix.*

Proof. Let (r, c) be the pivot and put $b_h = \sum_{i \in S} a_{ih}$ for each current column. Every b_h belongs to $\{0\} \cup \{2, 3, \ldots\}$. By Lemma 2.1, it is enough to test whether the sums *before the final entrywise saturation* can equal one.

First suppose $r \notin S$. For a surviving column h, the unsaturated new sum over S is

$$b_h + b_c a_{rh}. \tag{2}$$

Both terms are nonnegative, and each nonzero term is at least two. The new sum cannot be one. The set S itself is unchanged and nonempty.

Now suppose $r \in S$. Since $a_{rc} = 1$ and S is stopping, $b_c \geq 2$. In particular S has another row; otherwise b_c would equal one. Let $S' = S \setminus \{r\}$. For each surviving column h, its new unsaturated sum is

$$\sum_{i \in S'} (a_{ih} + a_{ic} a_{rh}) = b_h - a_{rh} + (b_c - 1) a_{rh} \tag{3}$$

$$= b_h + (b_c - 2) a_{rh}. \tag{4}$$

If $b_h = 0$, then $a_{rh} = 0$, so the result is zero. If $b_h \geq 2$, the second term is nonnegative, so the result is at least two. Again saturation cannot create a singleton sum. Thus S' is a nonempty stopping set. □

Merely saying that counts never decrease would not prove the first case: a previously zero column sum could in principle become one. Formula (2) excludes that possibility. Likewise, the pivoted row is deleted in the second case, so a monotonicity argument alone would be invalid. Formula (4) accounts for both its deletion and its positive replacements.

Corollary 3.2 (No full schedule crosses a stopping obstruction)**.** *If the initial matrix has a nonempty stopping set, no positive singleton schedule deletes all its rows.*

Proof. Apply Lemma 3.1 after every pivot. At least one row identity from the obstruction survives every finite prefix, so a full schedule is impossible. □

Lemma 3.3 (Canonical peeling residual)**.** *The union of any family of stopping sets is stopping. Every exhaustive private-column peeling order leaves the same row set $\mathcal{C}(A)$, namely the union of all stopping sets of the initial matrix.*

Proof. If a column has a positive entry on a union of stopping sets, that entry belongs to one member of the family. Its total on that member is at least two, hence so is its total on the union.

A private-column deletion cannot remove a row of any stopping set: its pivot column would then have total one on that set. Therefore every initial stopping set lies in the final residual. At termination no surviving column has total one on the residual. Every previously deleted column has total zero there, since it was private when deleted. Thus the residual is stopping with respect to *all original columns*. It contains every stopping set and is itself one, proving the assertion. □

The row set $\mathcal{C}(A)$ is canonical. A pivot schedule, the selected columns, and the surviving column basis need not be canonical. Nor does $|R \setminus \mathcal{C}(A)|$ generally equal the largest number of possible positive pivots.

4 The rectangular completeness theorem

Theorem 4.1 (Full cancellation without search)**.** *For a nonnegative integer $m \times n$ matrix A, the following conditions are equivalent:*

(i) a full positive singleton schedule exists;

(ii) every exhaustive private-column peeling order removes every row;

(iii) every nonempty subset of rows has a private column;

(iv) there is no nonempty weighted stopping set;

(v) some m columns give an $m \times m$ submatrix of permanent one.

The empty matrix satisfies these conditions with the convention that the permanent of the empty matrix is one.

Proof. Conditions (iii) and (iv) are logical complements of the defining obstruction. Condition (iii) implies (ii): if any row remains, the current row set has a private column; previously used columns have zero entries on it. Condition (ii) implies (i), because every private-column deletion is a legal positive pivot with no fill. Corollary 3.2 proves (i)⇒(iv).

For (ii)⇒(v), write the full peeling schedule as $(r_1, c_1), \ldots, (r_m, c_m)$. The selected square matrix, with rows and columns in these orders, has diagonal entries one and entries below the diagonal zero. Indeed, column c_i was private among $r_i, \ldots, r_m$. It is therefore upper triangular with permanent one.

Conversely, suppose the selected square matrix has permanent one. Its nonnegative integer permanent has exactly one nonzero permutation term, and each entry used in that term equals one. Relabel the selected columns c_i so that this matching is diagonal. Form a directed graph on the rows with edge $i \to j$, for $i \neq j$, when $a_{i,c_j} > 0$. A directed cycle would give a second nonzero

permutation term, by rotating the matching along that cycle. Thus the directed graph is acyclic. Every nonempty induced subgraph has a source j. The column c_j then has entry one in row j and zero in every other row of the chosen subset. This is (iii). □

The matching interpretation is a row-saturating uniquely restricted matching with unit matched entries. An entry of two is a genuine weight obstruction; replacing every positive entry by one would change the theorem. No permanent is computed by the algorithm. The permanent condition is a structural characterization and a small-instance independent oracle.

Corollary 4.2 (Square row-leaf criterion)**.** *For a square matrix, full positive elimination is equivalent to complete row-leaf peeling, where a leaf row has one nonzero entry and that entry is one.*

Proof. A square full-success matrix has permanent one by Theorem 4.1. The acyclic matching graph in the proof has a sink, whose row has only its matched unit entry. Deleting that row and its column preserves the unique matching. Induction gives a row-leaf schedule. The converse follows because a row-leaf pivot has an empty donor row and therefore no fill. □

This recovers the maintained square special case [6] but supplies the missing general rectangular decision criterion. It also shows why a square early-refusal optimization may safely remain in the proposed native integration.

5 What the native minimum-fill rule already decides

The inspected producer scores every current unit entry (r, c) by

$$(\rho_r - 1)(\kappa_c - 1), \qquad \rho_r = \sum_h a_{rh}, \quad \kappa_c = \sum_i a_{ic}, \tag{5}$$

and breaks ties by row size and labels [6]. The sums are ordinary sums of saturated entries.

Lemma 5.1 (Zero-fill moves preserve full solvability)**.** *If A has a full positive schedule, deleting a private column and its unique row, or deleting a leaf row and its column, leaves a matrix with a full positive schedule.*

Proof. Both kinds of pivot perform no fill. For a private-column pivot, any stopping set in the residual would already be stopping in the original matrix, since it has zero incidence with the removed column.

For a leaf-row pivot (r, c), suppose the residual has a nonempty stopping set S. All other columns already have total zero or at least two on S in the original matrix. If the total of column c on S is not one, then S is an original stopping set. If it is one, then $S \cup \{r\}$ is an original stopping set: the leaf row adds one to column c and zero to all others. Either case contradicts Theorem 4.1. □

Theorem 5.2 (Completeness of the inspected full-success heuristic)**.** *The minimum-score rule* (5)*, with any tie-breaking among minimum-score unit entries, deletes every row if and only if a full positive singleton schedule exists. On every successful run, all selected pivots have score zero and the number of positive fill updates is zero.*

Proof. If full elimination is possible, a private column exists by Theorem 4.1. Its score is zero. Scores are nonnegative, so every minimum-score pivot has score zero, meaning either $\rho_r = 1$ or $\kappa_c = 1$. These are exactly a leaf row or a private column. Lemma 5.1 preserves full solvability after such a pivot. Induction proves success with no fill at every step. The reverse implication holds because the algorithm's successful trace is itself a full positive singleton schedule. □

The documentation correction. Calling this minimum-fill rule "incomplete" is correct as a statement about recognizing arbitrary handlebodies, about arbitrary geometric moves, or about optimizing a partial schedule. It is not correct for the narrower decision "does a full positive singleton schedule exist for this fixed matrix?" The inspected implementation is complete for that decision. Consequently, private-column peeling cannot add a new full-success matrix that the current minimum-score rule rejects. It replaces repeated work and gives a canonical refusal certificate.

A genuine asymptotic improvement. For $A_m = [I_m \mid \mathbf{1}]$, the row-leaf pass stalls. At each subsequent minimum-fill stage, the remaining rows still have two nonzero entries. The two scans inside the minimum-score loop therefore inspect

$$4\sum_{j=1}^{m} j = 2m(m+1) \tag{6}$$

entries, although no fill is ever produced. Private-column peeling visits $2m$ nonzero incidences on deletion, after linear initialization. This separates repeated quadratic scanning from linear incidence work without changing the answer.

6 Linear certificates and independent verification

6.1 The producer

For each column keep the total of its current saturated entries and the XOR of the compact row indices of its nonzero incidences. A total of one implies that the XOR names its unique owner. Put all total-one columns in a FIFO queue. When such a column is processed, delete its owner row, subtract its entries from the affected column totals, update the XORs, and enqueue any column that has just reached total one. Stale queue entries are ignored.

The XOR is over row identities, not over multiplicities. A positive entry of two contributes its row identity once to the XOR and contributes two to the total. The owner is used only when the total is exactly one, so this distinction cannot introduce ambiguity.

Proposition 6.1 (Incidence bound)**.** *On compact array-indexed labels, the producer uses $O(m+n+s)$ word operations and storage. It deletes each incidence at most once and enqueues each column at most once. Column totals need only $O(\log(m+1))$ bits after saturation.*

Proof. Each row is deleted at most once and its adjacency list is visited only on deletion. Column totals decrease monotonically and can equal one for the first time at most once. A total is at most $2m$. Initialization and final residual collection require linear additional work. □

The Python reference supports arbitrary nonnegative integer labels and uses dictionaries. Its map operations have the usual expected/amortized costs; the unconditional deterministic linear

claim is for compact indexed adjacency arrays. Reading binary input integers is charged to input bit length. A deterministic comparison-based label normalization can be added with a logarithmic factor. The integration deliberately preserves the existing heap-based row-leaf fast path, whose ordering adds its own $O(m \log(m+1))$ term; the underlying column-only operator does not require a heap.

6.2 A timestamp proof

A certificate contains the schedule and the residual row list. Write $t(r)$ for the deletion time of a scheduled row and ∞ for a residual row. Each scheduled column c_j is paired with its pivot row r_j.

Proposition 6.2 (Independent sparse verification)**.** *The following checks verify an exhaustive private-column certificate:*

(a) scheduled rows and columns are distinct, every row is either scheduled or residual, and $a_{r_j c_j} = 1$;

(b) for every original nonzero a_{ic_j}, $t(i) \le j$, with equality only for $i = r_j$;

(c) every original column has residual total different from one.

They require $O(m+n+s)$ incidence work. The certificate itself has $O(m \log(m+n+1))$ bits on compact labels, excluding an external source identifier.

Proof. At step j, condition (b) says that every other row incident with column c_j was deleted earlier. Hence the pivot is private, and no fill has affected the surviving source rows. Induction authenticates the whole trace directly from the original matrix. Condition (c) says that the residual is stopping; earlier pivot columns have zero incidence with it by (b). Thus the trace is exhaustive and Lemma 3.3 identifies its residual as the maximal stopping set. Every check can be performed in a single sparse incidence pass after constructing the timestamp tables. □

A negative certificate can be even smaller: any nonempty stopping set suffices to exclude full positive elimination. It must be checked against all original columns, not merely whichever columns an untrusted producer elects to show. A prefix together with its stopping residual additionally authenticates the canonical closure.

6.3 Backward-compatible native replay

The new schedule uses the existing pairs $[r, c]$. A current general positive-pivot verifier already accepts it. The proposed core-verifier fast path first tests the private-column timestamp property on its *own source-reconstructed matrix*. If the test succeeds, it skips dense fill replay. If it fails, the existing general replay remains available. Thus legacy non-private certificates are not rejected merely because they lack the new fast property. No producer count summary replaces source reconstruction.

7 Implicit bundle columns and the topological conclusion

Proposition 7.1 (Uniformly nonunit columns may remain implicit)**.** *Partition the columns into a retained set C_0 and an omitted set C_∞. Suppose every entry in an omitted column is initially zero or at least two. Then:*

(i) omitted entries can never become one under positive pivots;

(ii) every legal pivot uses a retained column, and projection to C_0 commutes with every such update;

(iii) full positive eliminability and the maximal stopping row set are the same for the full matrix and the retained projection.

This holds for any finite number of omitted columns, including a number represented only by a binary multiplicity.

Proof. A zero or at-least-two entry plus a nonnegative multiple of another zero or at-least-two entry remains in the same set after saturation. This proves (i). A retained update involves only retained indices and the retained pivot column, proving (ii). An omitted column never has total one on any row subset, so it places no additional restriction on the stopping-set criterion in Theorem 4.1; this proves (iii). □

The hypothesis is columnwise and geometric. It is not enough to know a column total is initially at least two: two separate entries of one may later expose a pivot. Nor may a compressed family containing one exceptional singleton be treated as uniformly nonunit.

7.1 Application to the maintained boundary kernel

The source construction for bordered interval bundles retains dedicated boundary columns and omits genus columns that are initially zero or two. Connecting cycle columns in the projected model have the same nonunit property [8]. Proposition 7.1 therefore supplies an exact matrix-level decision theorem for that finite projection. Along a full private-column schedule, every other active row has zero in the pivot column, so not even omitted entries of active rows receive fill. Their binary handle counts remain bookkeeping data rather than expanded matrices.

If the source kernel has $O(T+P)$ retained rows and columns, its matrix has at most $O((T+P)^2)$ incidences. The new scheduling and timestamp stages require $O(T+P+s)$ incidence work after source preparation, instead of a general cubic saturated replay. This statement does not replace the cost of constructing the source geometry, authenticating its ordered attachment charts, or computing binary genus profiles.

Proposition 7.2 (Source-authenticated no-fill cancellation)**.** *Suppose a compact orientable three-manifold is given by an authenticated zero-, one-, and two-handle structure with no three-handles. Contract an authenticated spanning forest of the zero/one-handle graph, and form the actual unsigned cocore-intersection matrix. A full private-column schedule gives a geometric one/two-handle cancellation sequence. Every resulting connected component is a handlebody.*

Proof. At a private pivot, the chosen two-handle attaching circle crosses the chosen one-handle cocore exactly once, and every other remaining two-handle attaching circle avoids that cocore. The standard local cancellation removes this one/two pair: in a neighborhood of the unique crossing, the attaching two-handle cancels the one-handle, and no other attaching curve needs a passage across the removed cocore. In particular no positive replacements are required in the other two-handle rows. Repeat with the remaining authenticated structure.

After a full schedule there are only zero- and one-handles. Forest contraction left one zero-handle per connected component. An orientable connected manifold built from a ball by one-handles is a handlebody. Its genus is the number of surviving loop one-handles. □

For a projected source model, include all implicit one-handles in this final count. Equivalently, the maintained whole-cut census is obtained from initial zero/one genus, bundle genera and connecting graph cycle rank, minus the number of removed two-handles [8]. A genus-zero accepted component is a ball because of the geometric cancellation proof, not merely because its Euler characteristic is one.

Information not supplied by this proposition. An arbitrary integer matrix is not a handle structure. A successful census is not an explicit transported boundary pattern. Patterned hierarchy steps still require source-bound attaching maps and compatible markings. A refusal is not evidence of knottedness or of non-handlebodyness. These distinctions remain in force after the faster matrix theorem.

8 Optimal partial reduction by deferred rows

Example 8.1 (A stalled closure need not maximize pivots)**.** For

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix},$$

private-column peeling deletes no row. Nevertheless one positive pivot is legal and leaves (2). Thus the canonical stopping core is not an optimal partial-cancellation kernel.

Let $D \subseteq R$ be a set of *deferred* rows, and put $U = R \setminus D$. Deferred rows are never pivoted; they remain part of the represented handle structure and undergo the positive updates caused by pivots in U.

Theorem 8.2 (Exact deferral characterization)**.** *All rows in U can be eliminated without pivoting a row in D if and only if the row-restricted matrix $A[U, C]$ has empty stopping core. Equivalently, D meets every nonempty stopping set of the original matrix.*

Proof. For a schedule using only rows of U, every update between two surviving rows of U depends only on entries of those rows and the pivot row, which also belongs to U. Thus restriction to U commutes with the entire schedule. A schedule deleting exactly U yields a full positive schedule on $A[U, C]$, and Theorem 4.1 supplies a private-column schedule on that restriction.

Conversely, a private-column schedule on $A[U, C]$ is a legal positive schedule on the full matrix. Its pivot entries remain one in active rows. Entries in deferred rows may grow, but cannot change any active-row entry. This eliminates U while retaining D.

A nonempty subset $S \subseteq U$ is stopping in the restriction exactly when it is stopping in the original matrix: the definition sums only rows in S. Therefore the absence of such an S is equivalent to D meeting every original stopping obstruction. □

Definition 8.3 (Deferral number)**.** Define

$$\delta(A) = \min\{|D| : A[R \setminus D, C] \text{ has empty stopping core}\}.$$

The minimum exists because all rows may be deferred.

Corollary 8.4 (Maximum positive schedule length)**.** *The maximum number of positive singleton pivots achievable from A equals $m - \delta(A)$.*

Proof. For any schedule, let U be its pivoted row identities and D the remaining identities. Theorem 8.2 implies $|D| \geq \delta(A)$. Conversely, an optimal deferred set gives a schedule with exactly $m - \delta(A)$ pivots. □

Theorem 8.5 (Complete small-deferral search)**.** *For any $k \geq 0$, deciding whether at most k rows need be deferred, and producing a longest possible schedule when the answer is yes, takes*

$$\mathrm{poly}(\lambda) \sum_{j=0}^{\min(k,m)} \binom{m}{j} \tag{7}$$

bit operations, where λ is the encoded matrix length. On compact saturated adjacency data, the peeling work per subset is $O(m + n + s)$. The final retained rows can be transported in polynomial encoded time as in Section 9.

Proof. Enumerate deferred subsets in increasing size and peel the complementary rows. The first successful subset has size $\delta(A)$, so Corollary 8.4 proves optimality of the resulting length. Theorem 8.2 proves completeness. The number of subsets and the cost of each test give (7). If no subset succeeds, there is no schedule leaving at most k rows. □

In particular, for $k = O(\log \lambda)$, this local problem has a $\lambda^{O(\log \lambda)}$ algorithm. This is an unconditional parameterized statement about the specified calculus. It does not assert that every useful normal cut has logarithmic deferral, or that the deferred handles can be geometrically resolved at that cost.

8.1 Obstruction-directed branching and lower bounds

An alternative search node has already selected a deferred set D_0. Peel $A[R \setminus D_0, C]$. If a nonempty stopping set S remains, every successful extension must defer at least one row of S. Branch on these possibilities. With memoization of deferred sets, this is a complete obstruction-directed version of the same search. If a certified obstruction of size at most b can always be supplied at unsuccessful search nodes, the search tree has $O(b^k)$ leaves, with polynomial work per node. Such a bound on supplied obstruction size is an additional parameter hypothesis, not a consequence of being a stopping set.

Pairwise disjoint stopping obstructions give a simple lower bound: any deferred set must contain at least one row from each. A certificate listing q pairwise disjoint nonempty stopping sets proves $\delta(A) \geq q$. This lower bound is sharp on the three-ball family in Section 10.

9 Exact passive transport without expanded words

The deferral theorem is more useful when the retained rows are transported rather than simply named. Here we distinguish saturated decision data from exact nonnegative occurrence counts.

Let a private-column schedule on the active rows U be $(r_1, c_1), \ldots, (r_u, c_u)$. In that row and column order, the *unsaturated* active matrix has the form $[I + Q \mid W]$, where Q is strictly upper triangular and the columns of W are the unpivoted columns. This follows because later active rows have zero entries in earlier pivot columns. Write the passive rows in matching column order as $[B \mid E]$.

Theorem 9.1 (Finite path-sum transport)**.** *For exact nonnegative positive updates, the passive matrix remaining after the schedule is*

$$E' = E + B(I - Q)^{-1}W = E + \sum_{j=0}^{u-1} BQ^j W. \tag{8}$$

It can be computed by sparse forward propagation in $O((k+1)(m+n+s))$ arithmetic operations when there are k passive rows. Saturating its result at two gives exactly the saturated positive replay.

Proof. Before pivot i, let β_i denote the current passive coefficient of column c_i. Its contribution to a later pivot column c_j is $\beta_i Q_{ij}$, and its contribution to the final columns is $\beta_i W_{i,*}$. Thus each passive row starts with coefficients $B_{*,i}$, propagates them along the directed acyclic graph of Q, and adds their weighted terminal contributions to E.

A directed path from an initial pivot column to a final column has a product of the corresponding edge weights. Summing all such paths gives the powers in (8). Strict upper triangularity implies $Q^u = 0$, so $(I - Q)^{-1}$ is exactly the displayed finite sum. No analytic convergence is involved.

The active donor rows themselves never receive updates. A forward implementation reads each original active incidence at most once for each passive row and performs no explicit matrix powering. Initialization and output bookkeeping are linear in the stored data, giving the claimed arithmetic bound. Finally, Lemma 2.1 says that saturation commutes with every addition and multiplication in this computation. □

The sign in $(I - Q)^{-1}$ denotes a positive geometric-series expansion. In particular, this is *not* the ordinary Schur complement of the block $I + Q$ in the original matrix. Confusing those formulas would introduce cancellations that the geometric occurrence calculus does not authorize.

Theorem 9.2 (Polynomial bit length of exact transport)**.** *For an active pivot row r_i, let*

$$\sigma_i = \sum_{h \neq c_i} a_{r_i h}$$

be its exact donor mass. If a passive row initially has total mass M_0, then every intermediate and final passive row has mass at most

$$M_0 \prod_{i=1}^{u} \max(1, \sigma_i). \tag{9}$$

If all original entries have at most B bits, all transported entries have

$$O((u+1)(B + \log(n+1)))$$

bits. Hence exact transport has polynomial bit complexity in the original explicit encoded matrix size, without expanding attaching words.

Proof. At a pivot, let M be the current mass of a passive row and f its pivot-column coefficient. Removing that coefficient and inserting the donor contributes

$$M' = M - f + f\sigma_i.$$

Since $0 \le f \le M$, this is at most $M \max(1, \sigma_i)$. Iterating proves (9). With B-bit entries, both M_0 and each σ_i are bounded by $n2^B$. Taking logarithms gives the stated bit bound, including zero rows by using $\log(M+1)$. Combining this bound with Theorem 9.1 gives polynomial bit complexity using ordinary exact integer arithmetic. □

A conservative explicit bound is $O((k+1)(m+n+s)\,\mathsf{M}(L))$ bit operations, in addition to parsing, when L is the preceding bit-length bound and $\mathsf{M}(L)$ bounds multiplication of L-bit integers. This is not intended as an optimized arithmetic estimate; it makes clear why exponential *values* do not force exponential *encoded* cost.

Example 9.3 (Exponentially many passages, linearly many bits)**.** Let active row r_i contain a unit in c_i, a two in c_{i+1} when $i < u$, and a unit in one final column z. Let one passive row initially contain only a unit in c_1. The schedule $c_1, \dots, c_u$ is private among active rows. Its final coefficient at z is

$$1 + 2 + \cdots + 2^{u-1} = 2^u - 1.$$

The source has $O(u)$ incidences, the final value has exactly u bits, and a literal repeated-splicing representation would have exponentially many passages. The supplied implementation checks this identity at $u = 4096$.

What has, and has not, been transported. The theorem transports unsigned occurrence counts and numerical predicates derived from them. It does not recover the cyclic ordering of an attaching curve, its embedded bands, or the full boundary pattern. Those objects need a source-authenticated word, chart, or compressed geometric representation. The exact theorem applies to explicitly described columns; a binary omitted family needs its own authenticated uniform transfer rule before an exact vector over that family can be inferred. The full-success omission theorem in Section 7 does not require such a vector.

10 A geometric barrier: balls with large stopping cores

A stopping obstruction is a certificate against the fixed positive calculus, not against the underlying manifold. This distinction is not a pathological concern restricted to arbitrary algebraic presentations.

Proposition 10.1 (A three-ball outside the full positive calculus)**.** *There is a geometric handle structure of a three-ball, with one zero-handle, two one-handles and two two-handles, whose unsigned cocore-intersection matrix is*

$$H = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}. \tag{10}$$

It has no full positive singleton schedule, although its determinant is one.

Proof. Start with the standard two canceling one/two-handle pairs attached to a ball. The two two-handle attaching circles cross their respective one-handle cocores once, giving the identity matrix. Slide the first two-handle over the second along a band in the zero-handle boundary. Then

slide the second over the new first. Choose the standard positive parallel bands, disjoint from the cocore boundaries except for the passages already in the attaching circles. The unsigned row counts are then

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \longrightarrow \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \longrightarrow \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}.$$

Geometrically, the bands connect the original attaching circles in the zero-handle boundary; a parallel copy of the donor circle supplies its passages through the one-handles. The bands create no new cocore crossings and no existing crossing is canceled. For a more explicit topological interpretation, use the standard genus-two Heegaard splitting of the three-sphere. The attaching circles bound a complete meridian disk system in the complementary handlebody. Each displayed band move is a slide of that disk system; cutting the complementary handlebody along the slid disks still gives a ball. Attaching their two-handle neighborhoods to the first handlebody therefore gives the three-sphere minus the interior of that ball, which is a ball. Equivalently, two-handle slides change the chosen disk system, not the resulting three-manifold. The two slides can be undone to recover the original canceling diagram.

This argument specifies geometric band moves. The associated positive words may be used as mnemonics, but an abstract free-group presentation alone would not justify the conclusion.

Both columns of H have total at least two on its two rows, so the full row set is stopping. Theorem 4.1 excludes a full positive schedule. Directly, every legal first pivot leaves the exact one-by-one matrix (3), hence saturated entry two. Finally $\det(H) = 1$. □

The example does not require signed cancellation hidden in the update: the two displayed rows can be taken as cyclically reduced positive attaching words. Conversely, the inverse band moves *do* leave the positive-only pivot language. That is exactly why they can undo the obstruction.

Theorem 10.2 (Linear deferral is compatible with three-ball topology)**.** *For every $q \geq 1$ there is a geometric handle structure of a three-ball whose post-forest matrix has $2q$ rows, $2q$ columns, $4q$ nonzero entries, entries at most two, and deferral number q.*

Proof. Take q copies of the handle structure in Proposition 10.1 and form their boundary connected sum along disks disjoint from all attaching regions. A boundary connected sum of finitely many balls is a ball: joining two balls along a boundary disk thickens a disk and gives another ball, and induction applies. Contract the connecting zero/one forest. The resulting matrix is block diagonal with q copies of H.

The two rows of each block form a stopping obstruction, and these q sets are disjoint. Theorem 8.2 implies $\delta \geq q$. In each block defer its second row and pivot its first row using the first column. These q pivots are mutually independent, so $\delta \leq q$. The size and entry bounds follow from the displayed block. □

This refutes any assertion that ball topology alone forces $\delta = O(\log m)$ for arbitrary geometric handle presentations. It does not refute a bound after specified geometric normalizations, nor a theorem for the narrower class of inherited source kernels actually produced by the normal-cut constructor. Establishing membership in that narrower class would require a genuine source adapter, which is not supplied for this family.

10.1 An even simpler unreduced-position warning

A longitude attaching a two-handle to a solid torus can be isotoped to introduce a small finger across the meridian cocore boundary and back. The curve stays embedded and isotopic to the original longitude, while its unsigned intersection count becomes three. The resulting manifold is still a ball, but the one-by-one saturated matrix is (2). Thus normalization of geometric position matters even before handle slides become relevant. The block family above is more informative because it does not rely on an immediate opposite-sign finger pair.

10.2 Fibonacci amplification as a controlled example

Continuing alternate positive row slides in the two-handle diagram produces Fibonacci-type counts while preserving the ball. After four slides the matrix is

$$\begin{pmatrix} 2 & 3 \\ 3 & 5 \end{pmatrix},$$

so every entry saturates to two and there is not even an initial singleton pivot. The known inverse slide history still certifies a return to the identity diagram. This illustrates the importance of source history; it does not establish polynomial-time geometric replay of an arbitrarily compressed slide history. The size and authentication of the transported bands must be charged separately.

11 Executable evidence and controlled measurements

11.1 Independent checks rather than a single implementation

The delivered code uses only the Python standard library. The producer imports no proof checker. The independent checker imports no producer. Exhaustive oracles use dense tuples, recursive search over every legal positive pivot, direct enumeration of square-minor permanents, and enumeration of every row subset. The native comparison consists of a clearly labeled transcription of the two inspected planner function bodies, not an assertion that the complete repository checkout was executed.

The 34 unit tests include malformed counts and certificates, Boolean-versus-integer rejection, duplicate pivots, zero rows, arbitrary labels, cancellation callbacks, omitted-column invariance, source-heuristic agreement, exact passive transport, and integration-helper checks. The integration transformations are exercised on explicit patch-target fixtures. The real production checkout was not available in the execution environment, and no patch was applied to it.

For each matrix in Table 1, the audit compares full positive search, permanent-one minors, column peeling, the native minimum-score transcription, sparse certificate verification, dense positive replay, and the union of all stopping sets. Every successful native run has zero recorded fill updates. In addition, 1,020 explicit stopping-set/first-pivot invariant cases pass, and 1,241 matrices agree between exhaustive maximum-pivot search and exact minimum-deferral search. These are finite checks of the statements proved above, not empirical substitutes for their general proofs.

Exact transport is checked on 400 randomized upper-triangular active systems, with both an uncapped dense oracle and a saturation comparison. The chain example reaches 4096 output bits. A separate large-binary test uses an 8193-bit donor value and verifies the exact product $(2^{8192}-1)2^{8192}(2^{8192}+1)$. No expanded attaching words are generated.

Rows	Columns	Alphabet size	Matrices	Full successes
0	0	3	1	1
1	3	3	27	19
2	2	3	81	10
2	3	3	729	204
2	4	3	6561	2818
3	2	3	729	0
3	3	3	19683	654
3	4	2	4096	2316

Table 1: Exhaustive matrix audit. The total is 31,907 matrices. The alphabet of size three is $\{0, 1, 2\}$; size two is $\{0, 1\}$. All decision methods agreed.

11.2 Paired producer and proof-query timings

The benchmark retains seven interleaved timing rounds for every case, alternating arm order. Small cases are repeated inside each round. Reported times are medians in milliseconds. The three main arms are:

(i) the inspected composition of row-leaf peeling followed, on refusal, by the minimum-score producer;

(ii) the proposed composition of the same row-leaf pass followed by private-column peeling;

(iii) a column-only query including independent sparse verification and JSON certificate serialization.

The third arm is not claimed to include source extraction, a native geometric verifier, or a complete knot decision. The raw JSON also records the column-only producer separately.

Family	m	Old prod.	Proposed	New query	Ratio
Private + common	1	0.0026	0.0032	0.0083	0.81×
Private + common	8	0.0297	0.0145	0.0244	2.05×
Private + common	64	1.3257	0.1031	0.1570	12.86×
Private + common	256	19.9630	0.4528	0.6459	44.09×
Private + common	1024	334.9500	1.9516	2.7321	171.63×
Identity	1	0.0012	0.0012	0.0068	1.01×
Identity	64	0.0492	0.0495	0.1193	0.99×
Identity	1024	0.9268	0.8911	1.9637	1.04×
Stopping cycle	8	0.0325	0.0105	0.0189	3.09×
Stopping cycle	64	1.4121	0.0712	0.1155	19.82×
Stopping cycle	256	20.9904	0.3187	0.4507	65.86×
Single unit	1	0.0013	0.0012	0.0075	1.03×

Table 2: Paired matrix-kernel measurements, in milliseconds. Ratio compares the old and proposed producer compositions. "New query" includes the column-only producer, independent matrix checking and evidence serialization; it is not a whole native query.

For the 1024-row private-plus-common family, the proposed composition has a median of approximately 1.952 ms versus 334.950 ms for the old composition, a measured ratio of about 172×. The

mathematical reason for the trend is (6), not a fitted exponent. The negative one-row private-plus-common result is retained: the proposed composition is slower there. The identity family shows essentially no systematic producer gain, as expected when the original leaf path already succeeds. The stopping-cycle family improves the full-refusal decision, but the new closure deliberately does not reproduce the old producer's partial schedule.

These measurements support a local implementation change and preserve regressions. They give no measured recognition improvement on difficult knot diagrams. Files under `results/` retain all samples, environment details, counts and logs.

12 Integration plan and scope of the delivered changes

The integration proposal is intentionally narrower than replacing every call to the general pivot planner.

Completion-oriented producers. In `core_handlebody.py` and `bundle_boundary_kernel.py`, the proposed changes replace only the fallback used to decide whether *all* rows can cancel. The legacy general planner function remains available to clients that require its partial trace. Successful schedules retain the existing pair format. Core certificates keep the accepted `capped` method label so the current general verifier remains a valid fallback.

An independent core-verifier fast path. The proposal adds a private-column timestamp check after source reconstruction in `core_handlebody_verify.py`. A passing check removes the scheduled labels without dense arithmetic. A failing check falls through to the old positive update, preserving the acceptance language of old certificates. The helper has no producer import.

The whole-cut boundary verifier. The delivered integration does not alter `bundle_boundary_kernel_verify.py`. Its source reconstruction and census path should receive a separately reviewed analogous optimization. Consequently the proposed whole-cut producer change alone does not remove a potentially cubic whole-cut verification cost. This is an explicit remaining engineering task, not an omitted term in the benchmark.

Pins and application safeguards. `integration/apply.py` is a default-dry-run applicator. It checks the Git blob hash of all three modified source files, prepares and compiles all replacement text, and refuses ambiguous patch anchors or pre-existing new modules. With an explicit `-write` it makes backups and writes the reviewed transformations. The tool was tested on patch fixtures, not on the full production checkout. Its source pins are documented in Appendix A; a newer source revision should be ported and reviewed rather than accepted by bypassing the hash check.

Source authentication and stable failure semantics. The sparse producer accepts only nonnegative integer count dictionaries. It does not accept abelianized words as substitutes for geometric occurrences. An empty stopping core is a full-success matrix result; a nonempty core is a proof of non-eliminability in the specified calculus. Neither result independently authenticates source topology. Existing `INCONCLUSIVE` semantics must remain unchanged for geometric consumers.

13 Further research questions

1. Compare and consolidate the incoming stopping-core manuscripts. The two named archive bodies could not be reviewed here. Establish precisely which invariant, rectangular completeness statement, and geometric examples they already contain. Preserve attribution and reconcile different proof languages before integrating another overlapping module.

2. Complete native source-paired benchmarks. Run the proposed producer composition and source-reconstructed fast check on the maintained normal-cut fixture corpus. Charge construction, forest contraction, matrix extraction, source proof replay, serialization, and genus census. Record the same accepted cuts and all refusals. The current matrix evidence is insufficient to forecast the net speedup of those complete queries.

3. Optimize the whole-cut verifier without weakening its trust boundary. Use timestamp checking on the independently reconstructed retained matrix in the boundary verifier. Prove that the bulk counts and componentwise census use precisely the same surviving handles. Preserve the original general replay for non-private legacy schedules. This is the most direct unimplemented extension of the present optimization.

4. Find geometric escape moves from a canonical stopping core. Once $\mathcal{C}(A) \neq \varnothing$, further positive-only schedule search cannot produce a full cancellation. Search instead for source-certified isotopies, inverse two-handle slides, changes of cocore systems, or geometric simplifications outside the positive calculus. Which such moves can be generated from a polynomial-size local chart catalogue, with independent verification?

5. Exploit the localization supplied by an obstruction certificate. A candidate geometric move need not affect every row. Can its support be confined to a small certified stopping subset or to a bounded interface around that subset? Theorem 10.2 shows that total core size is not uniformly small for arbitrary ball presentations, but disjoint local obstructions may admit independent local repairs.

6. Bound escape effort on actual inherited source kernels. Identify structural restrictions satisfied by the normal-cut exporter that are absent from arbitrary handle presentations. Then formulate and prove a bound on an appropriate escape parameter after normalization. A statement only about being a ball or having trivial homology cannot suffice. A logarithmic escape bound would be meaningful only together with polynomially bounded geometric move generation and chart transport.

7. Improve exact small-deferral search. Determine when minimal stopping obstructions have bounded size, or when the incidence graph allows a useful decomposition. Investigate memoized obstruction branching, disjoint-obstruction lower bounds, and exact optimization on bounded interfaces. The canonical maximum core is easy to find but may be a poor branching set.

8. Transport ordered geometry, not only count vectors. The finite resolvent in Theorem 9.1 suggests a shared acyclic representation of repeated substitutions. Construct a source-bound ordered

word or band grammar implementing the same DAG, and prove that its geometric realization and boundary markings remain valid. The count theorem alone cannot decide intersection order or embeddedness.

9. Extend exact transport to binary families. When omitted columns occur in source-authenticated uniform families, determine whether a finite family of transfer recipes suffices after partial cancellation. Count the refinement events and the bit sizes of family endpoints. Do not replace an exponentially large nonuniform boundary pattern by its aggregate genus.

10. Formalize the obstruction and timestamp theorems. The positive-pivot invariant is finite and elementary enough for a small proof-assistant development. Formalize the saturation identities, the two row-sum formulas, and the independent certificate semantics first. Connect these to the geometric cancellation theorem only through a separately stated source-authentication interface.

11. Account for hierarchy resets globally. Even a polynomial local operator and a small terminal kernel do not control repeated source changes. A complete quasi-polynomial analysis must bound the number and encoded size of reconstructed states, pattern transport, and useful-surface discovery or correction. The present result removes unnecessary positive-pivot search from that accounting; it does not supply the remaining global potential.

14 Conclusion

The finite positive cancellation problem has an exact rectangular solution: full success is equivalent to the absence of a weighted stopping set, and can be decided and certified with sparse no-fill peeling. The inspected minimum-fill producer already has the same full-success language; its unnecessary cost is repeated scanning. Small deferral leads to an exact local optimization problem, while a finite positive resolvent transports the retained numerical data with polynomial bit length.

The geometric counterfamily is equally important. A linearly large stopping obstruction can occur in a genuine three-ball presentation with bounded entries. Thus the next coverage improvement must use authenticated geometric changes outside the positive-only calculus, not a more elaborate ordering of the same pivots. The delivered proofs, tests and integration proposal make that separation explicit and executable.

A Source audit and reproducibility boundary

The reference revision is `8cb7d9d25ac8cb64a9d147304e3441e98e94c7a4`. Paths below are relative to `Topology/UnknotRecognition/`. Source content was inspected through the GitHub connector. A Git blob identifier is not a SHA-256 file checksum; the integration applicator computes the actual Git blob hash including its header.

Inspected path	Git blob identifier
`fast/fastunknot/core_handlebody.py`	`13935f9cfa9ecf70fa5966a2ceec58f0f3167ff7`

Inspected path	Git blob identifier
`fast/fastunknot/bundle_boundary_kernel.py`	`8caf8a7387d4b4532f43e45707ba18d57626625a`
`fast/fastunknot/core_handlebody_verify.py`	`9819307acca29c06789bb986a978c6eed0c83b36`
`synthesis/bundle_boundary_kernel.tex`	`b95499443a7c672afffba159b5350f3bda4874fa`
`synthesis/bundle_primitive_annuli.tex`	`79123c0b72afaa74d741443a16542b0d25ffc35d`

The reference planner file in this package is an explicit transcription of two functions and their required import. It is not labeled as a byte-identical complete source snapshot. The production test suite, native knot benchmarks, and real-checkout patch application are marked `NOT_RUN`. Incoming archive metadata was observed, but their binary contents were not successfully decoded. These limitations constrain performance and priority claims, not the mathematical proofs in Sections 3–10.

B Claim and evidence ledger

Claim	Status and supporting evidence
Exact full positive-elimination criterion	Proved, Theorem 4.1; exhaustive 31,907-matrix agreement.
Completeness of the inspected minimum-score rule	Proved, Theorem 5.2; all audited successful runs have zero fill.
Canonical stopping residual and sparse verifier	Proved, Lemma 3.3 and Proposition 6.2; independent implementation and mutation tests.
Optimal small-deferral reduction	Proved, Theorem 8.2 and Corollary 8.4; 1,241 optimum comparisons.
Exact passive transfer and polynomial bit size	Proved, Theorems 9.1–9.2; 400 random oracle comparisons and binary stress tests.
Balls can have linear deferral number	Proved by geometric handle slides and boundary connected sums, Theorem 10.2; no native normal-cut exporter claimed.
Native producer/verification integration	Hash-guarded proposal; patch-target fixtures and independent helper tested; full checkout application not run.
Native difficult-knot speedup	Not measured. Matrix timings must not be reported as this claim.
General quasi-polynomial unknot recognition	Not established. Local parameterized bounds are stated with their exact scope.

C Reproduction and minimal use

From the package root, run:

```
python -m unittest discover -s tests -v
```

```
python scripts/audit.py
python scripts/benchmark.py
sh build.sh
```

The first command runs 34 tests. The audit writes `results/exhaustive_audit.json`; the benchmark writes every timing sample, a CSV summary and the LaTeX table used in this manuscript. Measurements will vary with the execution environment.

A minimal positive certificate is:

```
from cancellation import column_cancellation_plan, verify_peeling_certificate
A = {0: {0: 1, 2: 1}, 1: {1: 1, 2: 1}}
p = column_cancellation_plan(A)
cert = dict(schema='positive-column-peeling-v1',
           cancellations=p['cancellations'],
           remaining_rows=p['remaining_rows'])
assert verify_peeling_certificate(A, cert, require_complete=True)
```

This is a matrix certificate only. A native consumer must reconstruct A from authenticated geometric source data before drawing the topological conclusion of Proposition 7.2.

References

[1] Mathematical Institute, University of Oxford. *Marc Lackenby announces a new unknot recognition algorithm that runs in quasi-polynomial time.* 2021. https://www.maths.ox.ac.uk/node/38304.

[2] Marc Lackenby. *Incompressible surfaces, hierarchies and unknot recognition.* arXiv:2607.23350v1, 25 July 2026. Especially Sections 9–10. https://arxiv.org/abs/2607.23350v1.

[3] Changyan Di, David Proietti, I. Emre Telatar, Thomas J. Richardson, and Rüdiger L. Urbanke. Finite-length analysis of low-density parity-check codes on the binary erasure channel. *IEEE Transactions on Information Theory* 48(6), 1570–1579, 2002. Definition 1.1 and Lemma 1.1. https://cnls.lanl.gov/~chertkov/Papers/02DPTRU.pdf.

[4] Airlie Chapman and Mehran Mesbahi. On strong structural controllability of networked systems: a constrained matching approach. *2013 American Control Conference*, 6141–6146, 2013. https://depts.washington.edu/uwrainlab/wordpress/wp-content/uploads/2019/02/On_Strong_Structural_Controllability_of_Networked_Systems_ACC13.pdf.

[5] Mustafa Cemil Coşkun, Joachim Neu, and Henry D. Pfister. *Successive cancellation inactivation decoding for modified Reed–Muller and eBCH codes.* arXiv:2004.05969, 2020; accepted at ISIT 2020. https://arxiv.org/abs/2004.05969.

[6] Vladimir Reshetnikov, ProveIt repository. `Topology/UnknotRecognition/fast/fastunknot/core_handlebody.py` and `core_handlebody_verify.py`. Revision `8cb7d9d25ac8cb64a9d147304e3441e98e94c7a4`, inspected 12 October 2026. https://github.com/VladimirReshetnikov/ProveIt.

[7] Vladimir Reshetnikov, ProveIt repository. `Topology/UnknotRecognition/fast/fastunknot/bundle_boundary_kernel.py`. Same pinned revision; see Appendix A.

[8] Vladimir Reshetnikov, ProveIt repository. `Topology/UnknotRecognition/synthesis/bundle_boundary_kernel.tex`. Same pinned revision. Finite projected geometric handle model and genus census.

[9] Vladimir Reshetnikov, ProveIt repository. `Topology/UnknotRecognition/synthesis/bundle_primitive_annuli.tex`. Same pinned revision. Source bundle primitivity and geometric gluing.